Maker Execution Realism

Improvements report · **crypto_MFT_arb_platform** (`src/crypto_stat_arb`) · 12 October 2026
Scope: queue-aware maker simulation, canonical trade events, latency/clock unification, C++ matching-engine integration, verification and sensitivity study.

229 / 229
crypto_stat_arb tests passing (baseline 199)

Bit-identical
C++ vs Python engine on 230,808 real events

Identical
live paper fills vs replay of the same session

84 runs
sensitivity sweep, 6 cycles, 0 sanity violations

1. Executive summary

The maker (passive, post-only) simulation previously ran in a separate “universe” from the rest of the platform: its own flat 10 ms latency, wall-clock timing, raw-dictionary trades and a Python fallback model, while the C++ matching engine had never been compiled or exercised. This session unified it with the platform’s existing abstractions and verified the result end to end.

- **One execution path:** canonical `MarketEvent` + new `TradeEvent` → one event clock (`SimulationClock`) → platform `LatencyModel` → venue arrival → C++ `mm::MatchingEngine` → print-driven fills → fill-report latency → strategy.
- **Queue position at venue arrival** (decision + strategy + risk + entry latency), cancel races, post-only rejects, delayed inventory knowledge and preserved queue priority are now modelled.
- **C++ engine integrated and corrected** (three queue-accounting bugs fixed), with a bit-identical Python mirror and parity tests.
- **Sensitivity:** adding 35 ms of market-data staleness reduces filled quantity by 3.8% (Binance) / 4.5% (Bybit) and 75 ms by ~9%, while per-fill markouts are unchanged (±0.02 bps). The results are **not** produced by stale-book optimism.
- **Economic finding:** at-touch passive quoting loses ~1 bps per fill to adverse selection plus a 2 bps maker fee — negative in all 28 aggregated scenarios.

2. Problems found at the start

#	Problem	Impact
1	`maker_sim.py` used `latency_ms=10` independently of the platform `LatencyModel` (OMS used the `singapore` preset).	Maker and OMS simulations ran on different latency assumptions; Binance entry is ~39 ms in the preset.
2	Queue position was taken when the order was *decided*, not when it reached the venue.	Optimistic queue placement; no causal link to entry latency.
3	Trades were loaded as raw dicts outside the canonical pipeline; no `TradeEvent` type.	No validation, no shared envelope, special JSON only `maker_sim` understood.
4	Replay forced “book before trade” on equal receive timestamps.	Invented a universal exchange rule; could change queue states.
5	Live paper drove the maker with processing time, replay with receive time; vendored code monkey-patched `time.monotonic`.	Live and replay timelines guaranteed to diverge.
6	All 44 audited cycles ran `python_queue_model`; the C++ engine was never built for Python 3.12.	C++ path never exercised in this project.

#	Problem	Impact
7	Vendored `DryRunOrderManager` C++ path: book never fed to the engine; `uint64` fill ids looked up against string order ids; crashed (`'_CppFillEngine' has no attribute 'engine'`) once importable.	Every C++ trade-driven fill would have been dropped.
8	Random “price-touch decay” fills and a 35% adverse-selection gate.	Fills without any print behind them; not traceable.
9	Quotes cancelled and re-placed every second even at an unchanged price.	Queue priority discarded every interval.
10	C++ queue accounting: size-0 level erased our resting orders; shared-level FIFO volume wrongly consumed; trade + book delta could double count.	Lost orders, stuck queues, optimistic/pessimistic drift.

3. Improvements implemented

3.1 P0 – Core architecture

Gap	What was done
Latency parity	Maker builds its latency from the config `latency` section (same preset/overrides as the OMS) as a **separate instance with a derived seed**, so OMS draws are unchanged. Stage composition mirrors the OMS: decision *t* → dispatch *t*+strategy+risk → arrival +entry; cancel applies +cancel; fill reports +fill; acks/rejects +submit_ack. `maker_simulation.latency_ms` now raises an error; `maker_simulation.latency` overrides individual stages (e.g. 300 ns entry).
Queue at arrival	Orders join the C++ queue at venue arrival against the book at that instant. Cancels apply after cancel latency and the order can still fill meanwhile. Post-only orders crossing at arrival are rejected. Strategy inventory changes only when the fill report is observed; the inventory band uses that view plus all unfinished orders.
Canonical trades	New `models.TradeEvent` (sibling of `MarketEvent`, schema 1.1.0, reads legacy rows) and `EventType.TRADE`. `canonical.py` validates and dispatches both types with type-filtered sinks. Normalizers emit `TradeEvent` (`TradeTick` kept as alias). Live feed validates every trade envelope.
One event clock	New `sim_clock.SimulationClock`: `schedule`, `advance_to`, `drain`; deterministic (ts, sequence) order; timers due at an event fire before it; monotone time with regression counter. Live paper now uses each event’s `receive_ts_ns`.

3.2 P1 – Ordering, fills, cadence

Gap	What was done
Native ordering	Live feed stamps one `arrival_ordinal` across book and trade streams (written to `trace.jsonl` and `TradeEvent`). Replay reproduces the live interleaving from the trace (older traces: line order). Fallback key: receive time → exchange time → declared per-venue kind priority (`replay_tie_priority`) → event id. Native book/trade sequences are never compared. Duplicate ids fail closed to the fallback.
Print-driven fills	`maker_engine.py`: `CppQueueEngine` wraps the compiled engine with an explicit id map; `PythonQueueEngine` mirrors the arithmetic exactly. Every fill row records the causing `trade_event_id`, queue ahead at arrival and decision/dispatch/arrival/match/report timestamps. Book crossings without a print are counted (`crossed_without_print`), never filled.
Cadence vs latency	`quote_interval_ms` is a clock timer (strategy scheduling). Unchanged quotes keep resting and keep queue priority; moved quotes are cancelled and replaced.

3.3 C++ matching engine (`sim_engine/include/mm/matching_engine.hpp`)

- Compiled for arm64 / Python 3.12 (pybind11 3.1.0) into `sim_engine/`; now the default (`engine="auto"` → `cpp_mm_matching_engine`).
- Level sizes come only from the book feed; trades only burn queue ahead.

- Several own orders at one level share a print: only our own fills absorb volume.
- Book updates can only cap queue ahead; the capped amount is a `clamp_credit` so a print and its book delta count once in either arrival order.
- Print strictly through our price → full fill at our price. Size-0 level keeps our resting orders.
- New `set_queue_position_frac` (0–1, default 1.0 = back of visible queue).

3.4 Calibration and analysis features

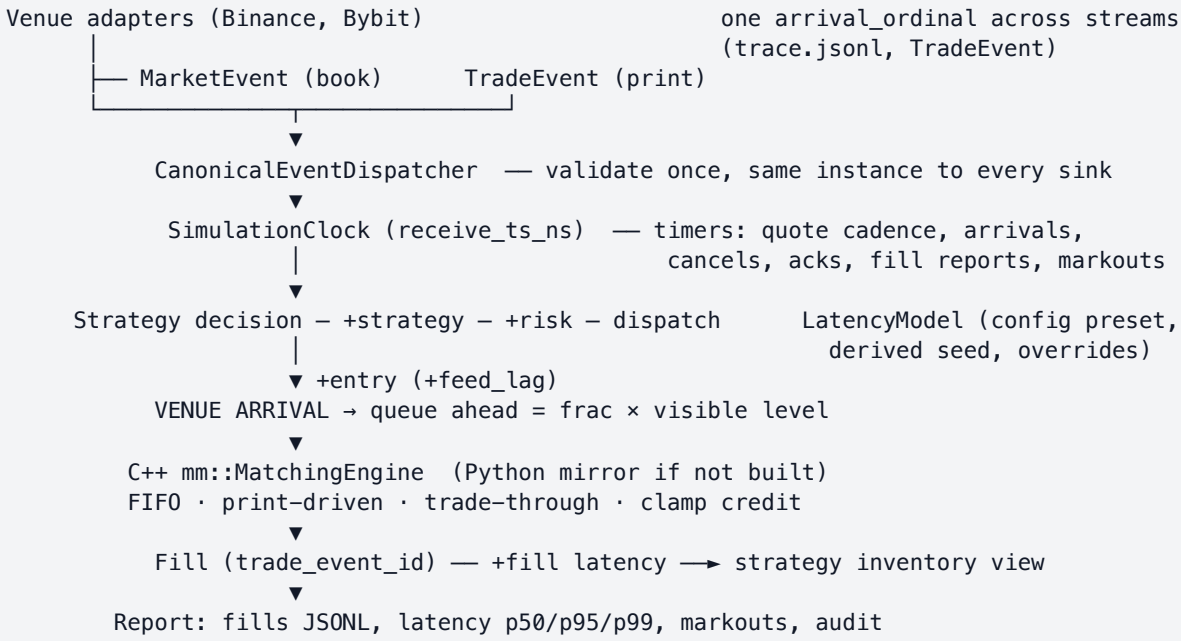
- `queue_position_frac` – share of the visible level ahead of a new order (old Python model hard-coded 0.7).
- `feed_lag_ms` – market-data staleness; venue instants are placed later on the receive-time clock.
- Per-fill markouts (`markout_horizons_ms`, default 1 s, 5 s) as an adverse-selection measure.
- `scripts/maker_sensitivity.py` – parallel sweep with CSV/JSON/Markdown output and `--from-csv` re-aggregation.
- Report schema 2.0.0 adds quotes kept, post-only rejects, cancel races, queue ahead, order states, latency percentiles; replay rejects invalid envelopes with counts; session audit adds latency source and an arrival-order check.

3.5 Files changed

File	Change
`maker_sim.py`	Rewritten (872 lines): event-clock simulator, OMS-parity latency, replay ordering, sweep helpers
`maker_engine.py`	New (248): C++ wrapper + bit-identical Python mirror
`sim_clock.py`	New (122): shared event-time clock
`models.py`	`TradeEvent`, `EventType.TRADE`, JSON support
`canonical.py`	Validation and type-aware dispatch for books and trades
`market_data_output.py`	Normalizers emit `TradeEvent`; `TradeTick` alias
`live_feed.py`	Shared `arrival_ordinal`; trace rows; trade envelope validation
`paper_live.py`	Maker driven by event receive time; `finish()`; fills JSONL
`session_audit.py`	Latency source metric; replay-order check; message update
`sim_engine/…/matching_engine.hpp`, `bindings.cpp`	Queue-accounting fixes, `clamp_credit`, `queue_position_frac`
`tests/test_maker_execution_path.py`	New: 30 tests (+ subtests)
`tests/test_watch_pipeline.py`	Fixture writes canonical trades
`scripts/maker_sensitivity.py`	New: sensitivity sweep
`config/paper_watch.json`	Removed obsolete `latency_ms`
`docs/execution_realism.md`, `.gitignore`s	Maker-simulation documentation, build steps; compiled module ignored

Package diff (`src/crypto_stat_arb`): 9 files changed, 1,014 insertions, 287 deletions, plus 2 new modules. Nothing has been committed.

4. Architecture after the changes



5. Test and verification results

5.1 Test suites

Suite / check	Result	Notes
crypto_stat_arb tests – baseline (before changes)	199 passed	Python 3.12, 11.8 s
crypto_stat_arb tests – after core changes	225 passed	+26 new tests
crypto_stat_arb tests – final	229 passed	9 subtests passed, 12.4 s
New module `test_maker_execution_path.py`	30 passed	listed in 5.2
platform_kit (trading_platform_poc) – before vs after	698 passed / 13 failed — identical	Failures pre-existing: missing `uv`, `tardis_dev`, `pyarrow`, `hftbacktest` on this host; 12 collection errors in both runs
Offline acceptance gate `scripts/run_all_local.py`	PASSED	225 tests, benchmark PASS, paper release id `bfbcab66…` unchanged (deterministic)
C++ engine build (cmake + pybind11 3.1.0)	Built, 0 errors	`mm_matching_engine.cpython-312-darwin.so`
Pre-change failures exposed by the build	2 failed → fixed	Vendored C++ path crashed once importable; resolved by moving off `DryRunOrderManager`

5.2 New tests (all passing)

Area	Test	Result
TradeEvent	legacy trade row reads as canonical trade	PASS
TradeEvent	normalizers emit canonical trades (Bybit cross sequence)	PASS
TradeEvent	validation rejects unknown aggressor side	PASS

Area	Test	Result
TradeEvent	dispatcher routes by type and shares the same instance	PASS
TradeEvent	recorder writes trades only when asked	PASS
SimulationClock	timers fire in order, before an event at the same time	PASS
SimulationClock	time never goes backwards (regressions counted)	PASS
SimulationClock	cancelled timer does not fire; drain lands the rest	PASS
Queue engine	semantics: FIFO shared print, zero-size level keeps order, trade-through	PASS (C++ & Python)
Queue engine	trade and its book delta burn the same queue in either order	PASS (C++ & Python)
Queue engine	queue_position_frac scales queue ahead at placement	PASS (C++ & Python)
Queue engine	C++ and Python engines bit-identical, 5,000 random ops × frac 1.0/0.7/0.0	PASS
Maker simulator	queue position assigned at venue arrival, not decision	PASS
Maker simulator	strategy inventory changes only when the fill report arrives	PASS
Maker simulator	order fills while its cancel is in flight	PASS
Maker simulator	a crossing book never fills without a print	PASS
Maker simulator	post-only order rejected if the book crossed it before arrival	PASS
Maker simulator	unchanged quote keeps its queue priority (no cancel/replace)	PASS
Maker simulator	feed lag takes the queue from the book received after arrival	PASS
Maker simulator	markouts are signed mid moves after each fill	PASS
Maker simulator	zero latency when latency is disabled is labelled	PASS
Config	maker latency = platform model on its own derived seed (OMS draws unchanged)	PASS
Config	stage overrides support latency sensitivity (300 ns entry)	PASS
Config	queue / feed-lag / markout settings read from config	PASS
Config	legacy maker_simulation.latency_ms is rejected	PASS
Replay order	arrival_ordinal reproduces the live interleaving	PASS
Replay order	tie priority is a declared per-venue policy	PASS
Replay order	exchange time breaks receive-time ties before event kind	PASS
Replay order	duplicate event ids fall back to receive-time ordering	PASS
End-to-end	live paper and its replay produce identical fills	PASS

5.3 Verification on real recorded data

Check	Data	Result
C++ vs Python engine parity	cycle-0006: 79,365 book events + 151,443 trades	fills JSONL byte-identical
C++ queue semantics smoke test	trade-first vs book-first; two orders one level; size-0 level; trade-through	all as designed
Live paper vs replay parity	fake Binance + Bybit books and trades through `LivePaperSession`	identical fill lists
Replay ordering on real sessions	trace-based `arrival_ordinal`	arrival_ordinal used
Duplicate event ids in traces	all 46 recorded cycles	0 duplicates
Session audit `analyze-session`	cycle-0006	simulation: OK (was WARN)
CLI `simulate-maker`	cycle-0001	C++ engine, 0 rejected envelopes, 0 sanity violations

Check	Data	Result
Sensitivity sweep	84 runs × 6 cycles	**0 sanity violations**

The cycle-0006 audit still reports **backtest_replay: FAIL** (replay-vs-paper DIVERGED). This is the pre-existing OMS replay divergence and is unrelated to the maker work.

5.4 Before vs after on cycle-0006 (same recorded data)

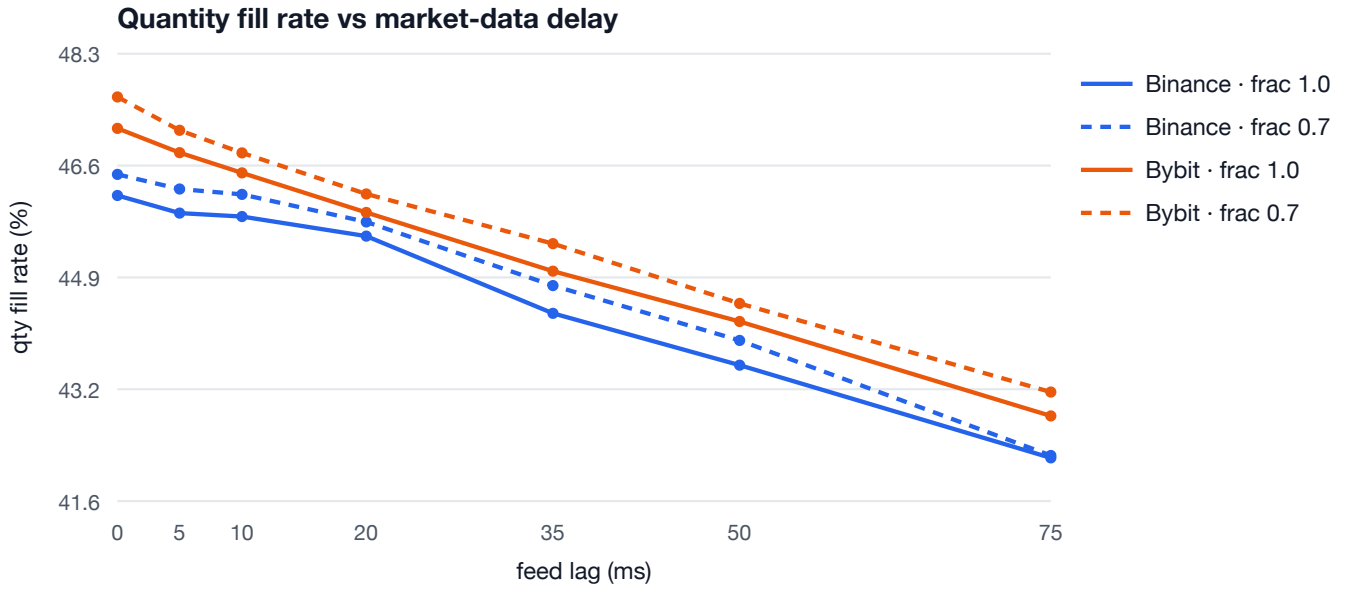
Metric	Before – Binance	After – Binance	Before – Bybit	After – Bybit
Engine	python_queue_model	cpp_mm_matching_engine	python_queue_model	cpp_mm_matching_engine
Latency	flat 10 ms	singapore preset (entry p50 39.1 ms)	flat 10 ms	singapore preset (entry p50 5.6 ms)
Orders sent (quotes)	2,739	1,132 (+1,787 kept resting)	3,088	1,513 (+1,625 kept)
Fill events	870	1,130 (378 through)	823	870 (504 through)
Post-only rejects	–	32	–	6
Fills while cancel pending	–	9	–	1
Mean queue ahead at arrival	–	3.576	–	0.504
Sanity violations	0	0	0	0

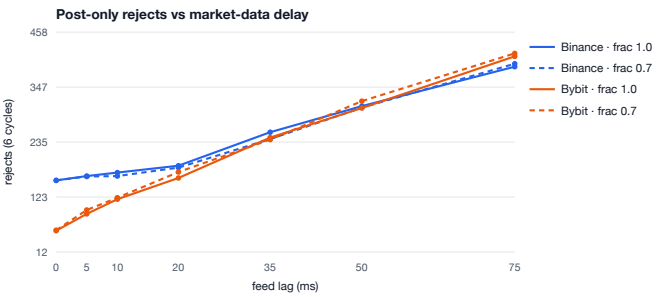
Old and new numbers are not directly comparable: “quotes” now means orders actually sent, the old model used a 0.7 queue fraction plus random touch fills, and unchanged quotes are no longer cancelled/replaced.

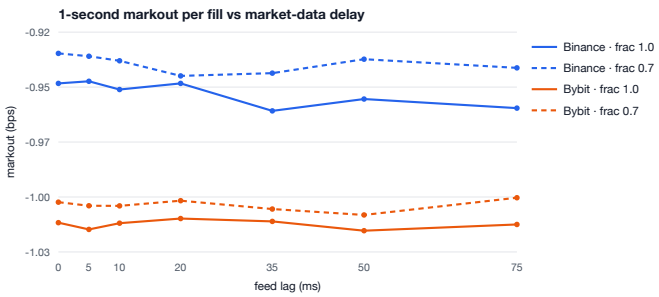
6. Sensitivity study: market-data delay × queue position

Question: how much of the simulated maker outcome survives when we remove the assumption that the observed book equals the venue book at order arrival?

Method: 84 deterministic replays on the C++ engine, ordering `arrival_ordinal`, cycles 0006, 0007, 0008, 0009, 0010, 0040 (≈30 min of live Binance + Bybit data each, 14–15 Sep 2026). Swept `feed_lag_ms` ∈ {0, 5, 10, 20, 35, 50, 75} and `queue_position_frac` ∈ {1.0, 0.7}. Quantity fill rate = filled qty / (orders sent × 0.001 BTC). Markouts: signed mid move after each fill (positive = favourable). PnL is mark-to-market incl. fees and inventory drift.







Findings

Finding	Evidence
Delay costs some fills, smoothly – no cliff	Filled qty –3.8% Binance / –4.5% Bybit at 35 ms; –8.5% / –9.2% at 75 ms (frac 1.0)
Per-fill economics unchanged → results not driven by stale-book optimism	1 s markout –0.94 → –0.96 bps (Binance), –1.01 → –1.01 (Bybit) from 0 to 75 ms
Queue position matters little for at-touch quotes	frac 0.7 vs 1.0 changes filled qty ~1%; 38% (Binance) and 57% (Bybit) of fills are trade-throughs
Post-only rejects are the strongest delay signature	Binance 157 → 388, Bybit 55 → 409 from 0 to 75 ms — best calibration target vs real testnet reject rate
At-touch passive harness is negative in every scenario	≈ –1 bps adverse selection per fill + 2 bps maker fee; fees ≈ 58% of the loss

Proposed acceptance criterion: at 35 ms market-data delay, filled quantity falls by ≤ 5% and per-fill markout shifts by ≤ 0.1 bps. Both venues pass (–3.8% / –4.5%; ≤ 0.02 bps).

Binance – queue_position_frac 1.0

Lag ms	Orders	Filled qty	Qty fill	Δ fill	Fully filled	Through	Queue ahead	MO 1s	MO 5s	PnL	Fees	PO rej.
0	5,386	2.4862	46.2%	+0.0%	46.2%	38.0%	3.429	-0.94	-1.29	-67.62	38.91	157
5	5,378	2.4682	45.9%	-0.6%	45.9%	38.1%	3.427	-0.94	-1.29	-67.42	38.63	166
10	5,386	2.4692	45.8%	-0.7%	45.9%	38.3%	3.429	-0.95	-1.30	-67.56	38.64	173
20	5,377	2.4492	45.5%	-1.3%	45.6%	38.4%	3.425	-0.94	-1.31	-67.30	38.33	187
35	5,387	2.3912	44.4%	-3.8%	44.4%	37.6%	3.381	-0.96	-1.31	-65.77	37.42	255
50	5,403	2.3562	43.6%	-5.5%	43.6%	37.7%	3.332	-0.95	-1.33	-64.48	36.88	308
75	5,425	2.2902	42.2%	-8.5%	42.2%	38.5%	3.251	-0.96	-1.35	-62.93	35.84	388

Binance – queue_position_frac 0.7

Lag ms	Orders	Filled qty	Qty fill	Δ fill	Fully filled	Through	Queue ahead	MO 1s	MO 5s	PnL	Fees	PO rej.
0	5,402	2.5107	46.5%	+0.0%	46.5%	37.8%	2.397	-0.93	-1.27	-68.35	39.29	157
5	5,395	2.4957	46.3%	-0.5%	46.3%	37.9%	2.400	-0.93	-1.27	-68.21	39.06	165
10	5,398	2.4927	46.2%	-0.6%	46.2%	38.4%	2.397	-0.93	-1.27	-68.26	39.01	166
20	5,377	2.4607	45.8%	-1.5%	45.8%	38.3%	2.399	-0.94	-1.28	-67.71	38.51	183
35	5,389	2.4145	44.8%	-3.6%	44.8%	38.5%	2.358	-0.94	-1.27	-66.46	37.79	241
50	5,417	2.3825	44.0%	-5.4%	44.0%	39.5%	2.317	-0.93	-1.24	-65.27	37.29	305
75	5,430	2.2942	42.2%	-9.1%	42.3%	39.0%	2.280	-0.94	-1.25	-63.28	35.90	394

Bybit – queue_position_frac 1.0

Lag ms	Orders	Filled qty	Qty fill	Δ fill	Fully filled	Through	Queue ahead	MO 1s	MO 5s	PnL	Fees	PO rej.
0	6,835	3.2239	47.2%	+0.0%	47.1%	57.2%	0.458	-1.01	-1.23	-82.10	50.47	55
5	6,854	3.2079	46.8%	-0.8%	46.8%	57.4%	0.456	-1.02	-1.25	-81.18	50.22	89
10	6,845	3.1829	46.5%	-1.4%	46.5%	57.1%	0.456	-1.01	-1.25	-80.49	49.83	119
20	6,851	3.1449	45.9%	-2.7%	45.9%	57.5%	0.453	-1.01	-1.25	-79.68	49.24	162
35	6,880	3.0976	45.0%	-4.5%	45.0%	57.3%	0.448	-1.01	-1.26	-78.24	48.50	244
50	6,903	3.0556	44.3%	-6.2%	44.2%	57.5%	0.443	-1.02	-1.25	-76.75	47.84	304
75	6,905	2.9585	42.8%	-9.2%	42.8%	57.6%	0.435	-1.01	-1.24	-74.37	46.33	409

Bybit – queue_position_frac 0.7

Lag ms	Orders	Filled qty	Qty fill	Δ fill	Fully filled	Through	Queue ahead	MO 1s	MO 5s	PnL	Fees	PO rej.
0	6,844	3.2605	47.6%	+0.0%	47.6%	47.6%	0.321	-1.00	-1.24	-82.61	51.04	56
5	6,840	3.2245	47.1%	-1.0%	47.1%	47.7%	0.320	-1.00	-1.25	-81.82	50.48	97
10	6,845	3.2035	46.8%	-1.8%	46.8%	47.8%	0.320	-1.00	-1.24	-81.02	50.15	122
20	6,850	3.1635	46.2%	-3.1%	46.1%	48.0%	0.318	-1.00	-1.23	-80.03	49.53	174
35	6,887	3.1292	45.4%	-4.6%	45.4%	47.8%	0.313	-1.01	-1.24	-78.89	48.99	240
50	6,898	3.0722	44.5%	-6.5%	44.5%	48.3%	0.310	-1.01	-1.25	-77.19	48.10	318
75	6,920	2.9897	43.2%	-9.3%	43.2%	48.5%	0.305	-1.00	-1.24	-74.90	46.81	415

Columns: Qty fill = filled qty ÷ offered qty; Δ fill vs 0 ms; Fully filled = orders filled completely ÷ orders sent; Through = share of fill events from prints beyond our price; MO = markout (bps); PO rej. = post-only rejects.

Totals over 6 cycles.

> An earlier version of this table counted fill *events*, which made frac 0.7 look worse; it fills the same or slightly more quantity in fewer, larger pieces. The tables above use quantity.

7. Known limitations

- **Queue calibration (P2) not done** – needs real order outcomes (testnet post-only orders).
- Default `feed_lag_ms = 0` keeps OMS parity; the sweep quantifies the bias rather than removing it.
- Book envelope does not carry `arrival_ordinal` (its JSON schema rejects extra fields); replay joins it from `trace.jsonl`.
- Maker report schema 2.0.0: "quotes" = orders sent; old and new cycle totals in `AUDIT_REPORT.md` are not comparable.
- Compiled module is arm64 / Python 3.12 and git-ignored; Docker/CI fall back to the Python mirror until a build step is added.
- FAST_MM's own copy of the C++ engine still has the queue bugs; vendored `dry_run_core.py` / `sim_fill_engine.py` are now unused.
- Sweep covers one market regime (14–15 Sep 2026); PnL includes inventory drift.

8. Recommended next steps

Priority	Item
P0	Build the C++ engine in Docker/CI; release path runs `engine=cpp` with a C++↔Python parity gate
P1	Move the matching engine to one shared component used by FAST_MM and crypto_stat_arb (port fixes once)
P1	Testnet calibration dataset: 100–500+ small post-only orders; fit queue survival and feed lag (post-only reject rate)
P2	Explicit report metric names (`orders_sent`, `orders_retained`) and legacy-schema flag in audits
P2	Extend the sweep to more regimes and add it as a permanent acceptance suite
P3	Archive unused vendored `dry_run_core.py` / `sim_fill_engine.py`

Appendix – reproduce

```
cd crypto_MFT_arb_platform
# build the C++ engine
python3.12 -m pip install pybind11
cmake -S src/crypto_stat_arb/sim_engine -B build/sim_engine -Dpybind11_DIR="$(python3.12 -m pybind11
--cmakedir)"
cmake --build build/sim_engine
# tests and acceptance
PYTHONPATH=src python3.12 -m pytest tests --ignore=tests/platform_kit -q
PYTHONPATH=src python3.12 scripts/run_all_local.py
# maker replay of a recorded cycle and the sensitivity sweep
PYTHONPATH=src python3.12 -m crypto_stat_arb.cli simulate-maker --config config/paper_watch.json \
    --session-dir runs/paper_watch/live/cycle-0006-20260914T142731Z --output runs/maker.json
python3.12 scripts/maker_sensitivity.py --top 6 --workers 6
```

Sweep artifacts: `runs/maker_sensitivity/20261011T172155Z/` (results.csv, summary.json, SENSITIVITY.md).